

Table of Contents

- 1 Overview of Machine Learning
- 2 Convex Optimization
- 3 Singular Value Decomposition
- 4 Fourier Analysis
- 5 Linear Regression
- 6 Dynamical Systems**

What is a Dynamical System?

A dynamical system describes the time evolution of a state variable.

Continuous Time:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x})$$

Discrete Time:

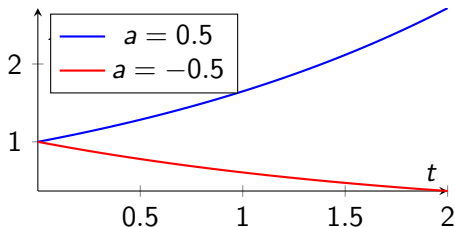
$$\mathbf{x}_{n+1} = \mathbf{f}(\mathbf{x}_n)$$

State Space: All possible values of $\mathbf{x} \in \mathbb{R}^n$

What is a Dynamical System?

Consider $\frac{dx}{dt} = ax$. Then:

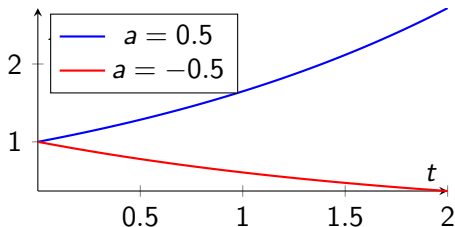
$$x(t) = x_0 e^{at}$$



What is a Dynamical System?

Consider $\frac{dx}{dt} = ax$. Then:

$$x(t) = x_0 e^{at}$$

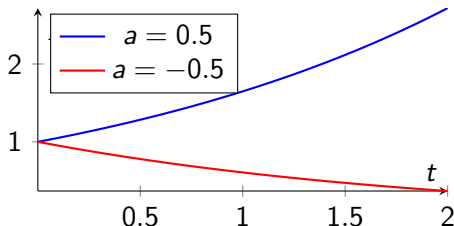


- Most differential equations cannot be solved analytically.

What is a Dynamical System?

Consider $\frac{dx}{dt} = ax$. Then:

$$x(t) = x_0 e^{at}$$



- Most differential equations cannot be solved analytically.
- Qualitative behavior of solutions.
 - Does the solution grow or decay or oscillate (stability)?
 - What is the behavior as $t \rightarrow \infty$?
 - How do initial conditions affect solutions?
 - ...

Fixed Points

A fixed point $\mathbf{x}^*$ satisfies:

Continuous: $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$

Discrete: $\mathbf{f}(\mathbf{x}^*) = \mathbf{x}^*$

Fixed Points

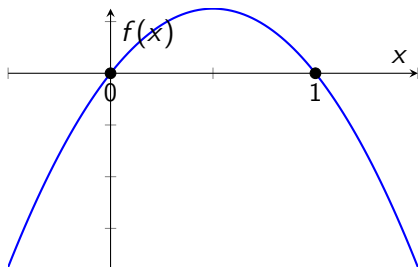
$$\frac{dx}{dt} = x(1 - x)$$

Fixed points: $x^* = 0, 1$

Fixed Points

$$\frac{dx}{dt} = x(1 - x)$$

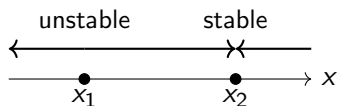
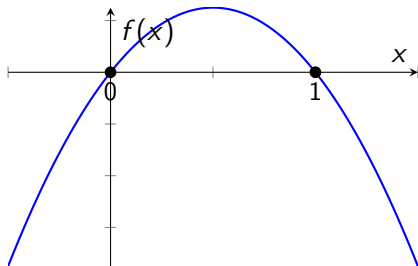
Fixed points: $x^* = 0, 1$



Fixed Points

$$\frac{dx}{dt} = x(1 - x)$$

Fixed points: $x^* = 0, 1$



1D Linear Stability Analysis

Let $\frac{dx}{dt} = f(x)$, and linearize near x^* :

$$f(x) \approx f(x^*) + f'(x^*)(x - x^*) \Rightarrow \frac{dx}{dt} \approx f'(x^*)(x - x^*)$$

1D Linear Stability Analysis

Let $\frac{dx}{dt} = f(x)$, and linearize near x^* :

$$f(x) \approx f(x^*) + f'(x^*)(x - x^*) \Rightarrow \frac{dx}{dt} \approx f'(x^*)(x - x^*)$$

Stability Criteria:

- $f'(x^*) < 0$: stable
- $f'(x^*) > 0$: unstable

Bifurcation

- A qualitative change in the dynamics as a parameter varies
- Study of how fixed points appear, disappear, or change stability

Bifurcation

Saddle-Node Bifurcation

$$\frac{dx}{dt} = \alpha + x^2$$

Bifurcation

Saddle-Node Bifurcation

$$\frac{dx}{dt} = \alpha + x^2$$

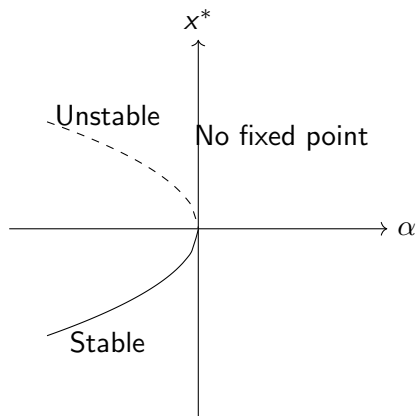
- $\alpha < 0$: Two fixed points
- $\alpha = 0$: One (semi-stable) fixed point
- $\alpha > 0$: No fixed point

Bifurcation

Saddle-Node Bifurcation

$$\frac{dx}{dt} = \alpha + x^2$$

- $\alpha < 0$: Two fixed points
- $\alpha = 0$: One (semi-stable) fixed point
- $\alpha > 0$: No fixed point



Bifurcation

Transcritical Bifurcation

$$\frac{dx}{dt} = \alpha x - x^2$$

Bifurcation

Transcritical Bifurcation

$$\frac{dx}{dt} = \alpha x - x^2$$

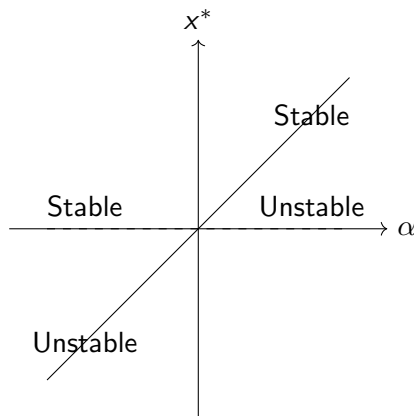
- Fixed points at $x = 0$ and $x = \alpha$
- They exchange stability at $\alpha = 0$

Bifurcation

Transcritical Bifurcation

$$\frac{dx}{dt} = \alpha x - x^2$$

- Fixed points at $x = 0$ and $x = \alpha$
- They exchange stability at $\alpha = 0$



Bifurcation

Pitchfork Bifurcation (Supercritical)

$$\frac{dx}{dt} = \alpha x - x^3$$

Bifurcation

Pitchfork Bifurcation (Supercritical)

$$\frac{dx}{dt} = \alpha x - x^3$$

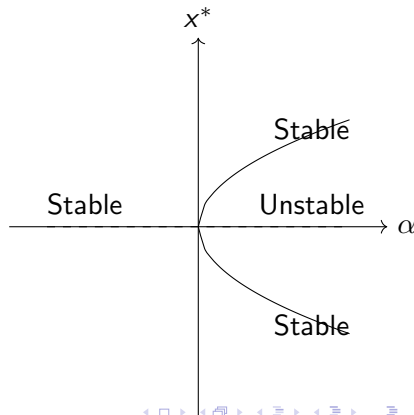
- $\alpha < 0$: One stable point
- $\alpha > 0$: Two new stable branches appear

Bifurcation

Pitchfork Bifurcation (Supercritical)

$$\frac{dx}{dt} = \alpha x - x^3$$

- $\alpha < 0$: One stable point
- $\alpha > 0$: Two new stable branches appear



The Logistic Map

$$x_{n+1} = rx_n(1 - x_n), \quad x_n \in [0, 1], \quad r \in [0, 4]$$

The Logistic Map

$$x_{n+1} = rx_n(1 - x_n), \quad x_n \in [0, 1], \quad r \in [0, 4]$$

Fixed points: Set $x_{n+1} = x_n = x^*$:

$$x^* = rx^*(1 - x^*) \Rightarrow x^* = 0 \quad \text{or} \quad x^* = 1 - \frac{1}{r}$$

The Logistic Map

$$x_{n+1} = rx_n(1 - x_n), \quad x_n \in [0, 1], \quad r \in [0, 4]$$

Fixed points: Set $x_{n+1} = x_n = x^*$:

$$x^* = rx^*(1 - x^*) \Rightarrow x^* = 0 \quad \text{or} \quad x^* = 1 - \frac{1}{r}$$

- $x_1^* = 0$: exists for all r
- $x_2^* = 1 - \frac{1}{r}$: only valid for $r > 1$

The Logistic Map

Linearize: $f(x) = rx(1 - x)$, $f'(x) = r(1 - 2x)$

The Logistic Map

Linearize: $f(x) = rx(1 - x)$, $f'(x) = r(1 - 2x)$

Stable if $|f'(x^*)| < 1$

The Logistic Map

Linearize: $f(x) = rx(1 - x)$, $f'(x) = r(1 - 2x)$

Stable if $|f'(x^*)| < 1$

- $x_1^* = 0$: stable if $0 < r < 1$
- $x_2^* = 1 - \frac{1}{r}$: stable if $1 < r < 3$

The Logistic Map

Period Doubling and Chaos

The Logistic Map

Period Doubling and Chaos

- For $r > 3$: bifurcations to period-2, period-4, etc.

The Logistic Map

Period Doubling and Chaos

- For $r > 3$: bifurcations to period-2, period-4, etc.
- Accumulation point at $r_\infty \approx 3.56995$
- Chaos beyond that: sensitive to initial conditions

The Logistic Map

Period Doubling and Chaos

- For $r > 3$: bifurcations to period-2, period-4, etc.
- Accumulation point at $r_\infty \approx 3.56995$
- Chaos beyond that: sensitive to initial conditions

Lyapunov Exponent

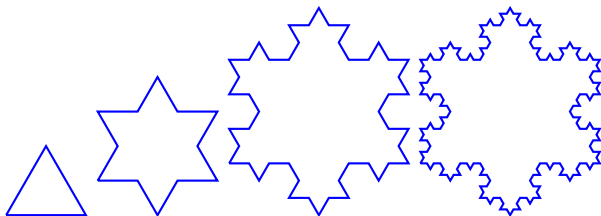
$$\lambda = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n \ln |f'(x_i)|$$

- $\lambda < 0$: stable fixed point or periodic orbit
- $\lambda > 0$: chaos

What is a Fractal?

Example: Koch Snowflake

- Start with an equilateral triangle.
- At each iteration, replace the middle third of each side with two segments forming a “bump”.
- Each iteration adds detail while preserving self-similarity.



What is a Fractal?

A fractal is a set with:

- Self-similarity at different scales.
- Fine structure everywhere.
- Non-integer dimension.

Fractal Dimensions

Fractal dimension quantifies the “complexity” or “roughness” of a fractal set.

- Traditional shapes: dimension = integer.
- Fractals: dimension = non-integer.

Fractal Dimensions

Fractal dimension quantifies the “complexity” or “roughness” of a fractal set.

- Traditional shapes: dimension = integer.
- Fractals: dimension = non-integer.
- Intuition: fractals fill space more than a line, less than an area.

Fractal Dimensions

To compute the dimension of a self-similar shape, we define the similarity dimension:

Fractal Dimensions

To compute the dimension of a self-similar shape, we define the similarity dimension:

For a self-similar shape made of N copies of itself, each scaled by a similarity with contraction factor s

$$N = s^{-D} \quad \Rightarrow \quad D = \frac{\log N}{\log(1/s)}$$

Where:

- N : number of self-similar pieces
- s : scaling factor for each piece

Fractal Dimensions

To compute the dimension of a self-similar shape, we define the similarity dimension:

For a self-similar shape made of N copies of itself, each scaled by a similarity with contraction factor s

$$N = s^{-D} \quad \Rightarrow \quad D = \frac{\log N}{\log(1/s)}$$

Where:

- N : number of self-similar pieces
- s : scaling factor for each piece

Example (Koch Snowflake):

- Each line segment is replaced by 4 segments of length $\frac{1}{3}$
- So: $N = 4$, $s = 1/3$

$$D = \frac{\log 4}{\log 3} \approx 1.2619$$

Fractal Dimensions

Box-Counting Dimension: Cover the fractal with boxes of size ε , and count how many are needed:

$$D_B = \lim_{\varepsilon \rightarrow 0} \frac{\log N(\varepsilon)}{\log(1/\varepsilon)}$$

- $N(\varepsilon)$: number of boxes of side ε needed to cover the set
- As $\varepsilon \rightarrow 0$, captures self-similar structure

Two Dimensional Dynamical Systems

$$\begin{cases} \dot{x} = f(x, y) \\ \dot{y} = g(x, y) \end{cases}$$

Two Dimensional Dynamical Systems

$$\begin{cases} \dot{x} = f(x, y) \\ \dot{y} = g(x, y) \end{cases}$$

- Describes the evolution of a point $(x(t), y(t)) \in \mathbb{R}^2$.

Two Dimensional Dynamical Systems

$$\begin{cases} \dot{x} = f(x, y) \\ \dot{y} = g(x, y) \end{cases}$$

- Describes the evolution of a point $(x(t), y(t)) \in \mathbb{R}^2$.
- Fixed point (equilibrium): $\dot{x} = \dot{y} = 0$.

Two Dimensional Dynamical Systems

$$\begin{cases} \dot{x} = f(x, y) \\ \dot{y} = g(x, y) \end{cases}$$

- Describes the evolution of a point $(x(t), y(t)) \in \mathbb{R}^2$.
- Fixed point (equilibrium): $\dot{x} = \dot{y} = 0$.
- Linearize system near each fixed point to analyze stability.

Two Dimensional Dynamical Systems

For a linear system:

$$\dot{\mathbf{x}} = A\mathbf{x}, \quad A \in \mathbb{R}^{2 \times 2}$$

Solution: $\mathbf{x}(t) = \mathbf{x}_0 e^{At}$

Two Dimensional Dynamical Systems

For a linear system:

$$\dot{\mathbf{x}} = A\mathbf{x}, \quad A \in \mathbb{R}^{2 \times 2}$$

Solution: $\mathbf{x}(t) = \mathbf{x}_0 e^{At}$

For a nonlinear system:

$$\dot{\mathbf{x}} = F(\mathbf{x})$$

Two Dimensional Dynamical Systems

For a linear system:

$$\dot{\mathbf{x}} = A\mathbf{x}, \quad A \in \mathbb{R}^{2 \times 2}$$

Solution: $\mathbf{x}(t) = \mathbf{x}_0 e^{At}$

For a nonlinear system:

$$\dot{\mathbf{x}} = F(\mathbf{x})$$

Linearized system:

$$\dot{\mathbf{u}} = J(\mathbf{x}^*)\mathbf{u}, \quad \text{where } J = \begin{bmatrix} \partial f / \partial x & \partial f / \partial y \\ \partial g / \partial x & \partial g / \partial y \end{bmatrix}_{(x^*, y^*)}$$

Two Dimensional Dynamical Systems

For a linear system:

$$\dot{\mathbf{x}} = A\mathbf{x}, \quad A \in \mathbb{R}^{2 \times 2}$$

Solution: $\mathbf{x}(t) = \mathbf{x}_0 e^{At}$

For a nonlinear system:

$$\dot{\mathbf{x}} = F(\mathbf{x})$$

Linearized system:

$$\dot{\mathbf{u}} = J(\mathbf{x}^*)\mathbf{u}, \quad \text{where } J = \begin{bmatrix} \partial f / \partial x & \partial f / \partial y \\ \partial g / \partial x & \partial g / \partial y \end{bmatrix}_{(x^*, y^*)}$$

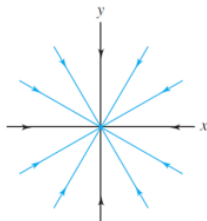
Classification based on eigenvalues λ_1, λ_2 of J :

- The sign of $\text{Re}(\lambda_{1,2})$
- Whether eigenvalues are real or complex

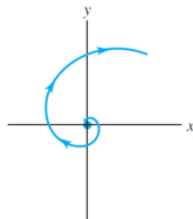
Fixed Point Classification

Eigenvalues	Type	Stability	Trajectory
Real, $\lambda_1, \lambda_2 < 0$	Node	Stable	Inward straight lines
Real, $\lambda_1, \lambda_2 > 0$	Node	Unstable	Outward straight lines
Real, opposite signs	Saddle	Unstable	Hyperbolic
Complex, $\text{Re}(\lambda) < 0$	Spiral sink	Stable	Inward spiral
Complex, $\text{Re}(\lambda) > 0$	Spiral source	Unstable	Outward spiral
Pure imaginary $\lambda = \pm i\omega$	Center	Neutral	Closed orbits

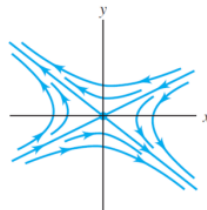
Fixed Point Classification



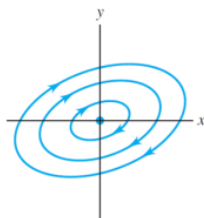
Node
(asymptotically stable)



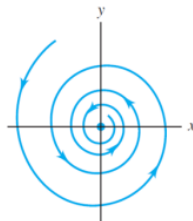
Spiral
(unstable)



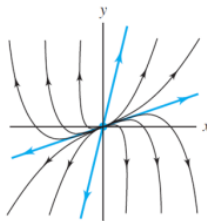
Saddle
(unstable)



Center
(stable)



Spiral
(asymptotically stable)



Node
(unstable)

Example: Pendulum

The pendulum equation:

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin \theta = 0$$

The first-order system:

$$\begin{cases} \dot{\theta} = \omega \\ \dot{\omega} = -\frac{g}{L} \sin \theta \end{cases}$$

Example: Pendulum

Pendulum Eigenvalue Analysis

Example: Pendulum

Pendulum Eigenvalue Analysis

The fixed points are:

$$(\theta^*, \omega^*) = (n\pi, 0), \quad n \in \mathbb{Z}$$

Example: Pendulum

Pendulum Eigenvalue Analysis

The fixed points are:

$$(\theta^*, \omega^*) = (n\pi, 0), \quad n \in \mathbb{Z}$$

Jacobian:

$$J = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} \cos \theta & 0 \end{bmatrix}$$

$$\det(J - \lambda I) = \lambda^2 + \omega_0^2 \cos \theta = 0 \quad (\omega_0 = \sqrt{\frac{g}{L}})$$

Example: Pendulum

Pendulum Eigenvalue Analysis

The fixed points are:

$$(\theta^*, \omega^*) = (n\pi, 0), \quad n \in \mathbb{Z}$$

Jacobian:

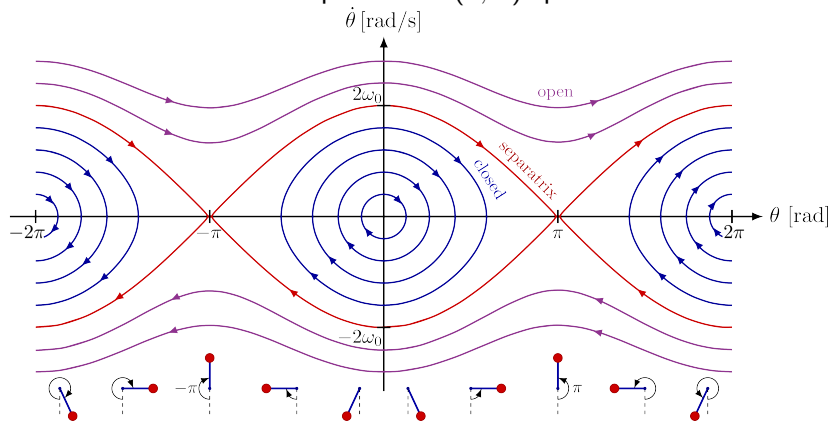
$$J = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} \cos \theta & 0 \end{bmatrix}$$

$$\det(J - \lambda I) = \lambda^2 + \omega_0^2 \cos \theta = 0 \quad (\omega_0 = \sqrt{\frac{g}{L}})$$

- $\lambda^2 + \omega_0^2 = 0 \Rightarrow \lambda = \pm i\omega_0 \Rightarrow$ **center**. ($n = \dots, -2, 0, 2, \dots$)
- $\lambda^2 - \omega_0^2 = 0 \Rightarrow \lambda = \pm \omega_0 \Rightarrow$ **saddle**. ($n = \dots, -1, 1, \dots$)

Example: Pendulum

Phase portrait in (θ, ω) space.



Limit Cycle

A **limit cycle** is an isolated closed trajectory in the phase plane.

- All nearby trajectories spiral toward it (stable), or away (unstable).
- Represents a self-sustained oscillation.
- Nonlinear systems.

Limit Cycle

Stability of Limit Cycles

- **Stable (attracting)**: all nearby trajectories approach the cycle
- **Unstable (repelling)**: all nearby trajectories move away
- **Semi-stable**: attract from one side, repel from the other

Example: Van der Pol Oscillator

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$$

Example: Van der Pol Oscillator

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$$

Convert to first-order system:

$$\begin{cases} \dot{x} = y \\ \dot{y} = \mu(1 - x^2)y - x \end{cases}$$

Example: Van der Pol Oscillator

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$$

Convert to first-order system:

$$\begin{cases} \dot{x} = y \\ \dot{y} = \mu(1 - x^2)y - x \end{cases}$$

Fixed points:

$$\begin{cases} \dot{x} = y = 0 \\ \dot{y} = \mu(1 - x^2)y - x = 0 \end{cases} \Rightarrow (x^*, y^*) = (0, 0)$$

Example: Van der Pol Oscillator

Linearize the system at the fixed point $(0,0)$ using the Jacobian matrix:

$$J(x, y) = \begin{bmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2\mu xy - 1 & \mu(1 - x^2) \end{bmatrix}$$

Example: Van der Pol Oscillator

Linearize the system at the fixed point $(0,0)$ using the Jacobian matrix:

$$J(x, y) = \begin{bmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2\mu xy - 1 & \mu(1 - x^2) \end{bmatrix}$$

$$J(0,0) = \begin{bmatrix} 0 & 1 \\ -1 & \mu \end{bmatrix}$$

Example: Van der Pol Oscillator

Linearize the system at the fixed point $(0,0)$ using the Jacobian matrix:

$$J(x, y) = \begin{bmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2\mu xy - 1 & \mu(1 - x^2) \end{bmatrix}$$

$$J(0,0) = \begin{bmatrix} 0 & 1 \\ -1 & \mu \end{bmatrix}$$

$$\lambda^2 - \mu\lambda + 1 = 0 \Rightarrow \lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

Example: Van der Pol Oscillator

$$\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

Example: Van der Pol Oscillator

$$\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

$\mu > 0$: The real part of the eigenvalues is $\text{Re}(\lambda) = \mu/2 > 0$.

Example: Van der Pol Oscillator

$$\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

$\mu > 0$: The real part of the eigenvalues is $\text{Re}(\lambda) = \mu/2 > 0$.

- If $0 < \mu < 2$, the eigenvalues are complex with a positive real part. The origin is an **unstable spiral source**.

Example: Van der Pol Oscillator

$$\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

$\mu > 0$: The real part of the eigenvalues is $\text{Re}(\lambda) = \mu/2 > 0$.

- If $0 < \mu < 2$, the eigenvalues are complex with a positive real part. The origin is an **unstable spiral source**.
- If $\mu \geq 2$, the eigenvalues are real and positive. The origin is an **unstable node**.

Example: Van der Pol Oscillator

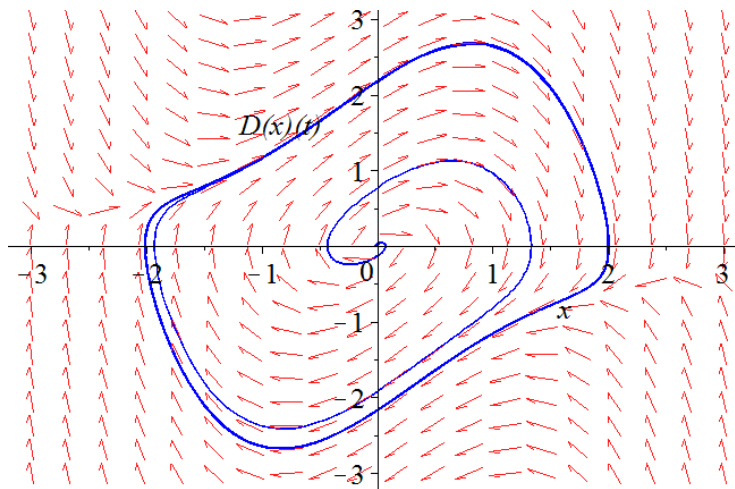
$$\lambda = \frac{\mu \pm \sqrt{\mu^2 - 4}}{2}$$

$\mu > 0$: The real part of the eigenvalues is $\text{Re}(\lambda) = \mu/2 > 0$.

- If $0 < \mu < 2$, the eigenvalues are complex with a positive real part. The origin is an **unstable spiral source**.
- If $\mu \geq 2$, the eigenvalues are real and positive. The origin is an **unstable node**.

For any $\mu > 0$, the origin is unstable. Trajectories spiral away from the origin and converge to a **stable limit cycle**, representing a self-sustained oscillation.

Example: Van der Pol Oscillator



Summary of What You've Learned Today

- Sparsity and Compressed Sensing

Summary of What You've Learned Today

- Sparsity and Compressed Sensing
- Fundamental Dynamical Systems
 - Fixed Points.
 - Stability Analysis.
 - Bifurcation.
 - Logistic Map.
 - Fractal.
 - Limit Cycle.

Summary of What You've Learned Today

- Sparsity and Compressed Sensing
- Fundamental Dynamical Systems
 - Fixed Points.
 - Stability Analysis.
 - Bifurcation.
 - Logistic Map.
 - Fractal.
 - Limit Cycle.

See you on Wednesday (7/16) for Lecture 7 on
Machine Learning Dynamical Systems.

Koopman Operator Theory

Koopman Operator Theory

Consider a discrete-time nonlinear dynamical system that evolves the state vector $x \in \mathbb{R}^n$ according to:

$$x_{k+1} = f(x_k)$$

Koopman Operator Theory

Consider a discrete-time nonlinear dynamical system that evolves the state vector $x \in \mathbb{R}^n$ according to:

$$x_{k+1} = f(x_k)$$

Whirlpool. The state x_k could be the position of every water molecule. The function f represents the complex, nonlinear laws of fluid dynamics.



Koopman Operator Theory

Consider a discrete-time nonlinear dynamical system that evolves the state vector $x \in \mathbb{R}^n$ according to:

$$x_{k+1} = f(x_k)$$

Whirlpool. The state x_k could be the position of every water molecule. The function f represents the complex, nonlinear laws of fluid dynamics.



Instead of studying state evolution, the Koopman operator $\mathcal{K}$ acts on **observable functions**, $g : \mathbb{R}^n \rightarrow \mathbb{R}$. It advances observables linearly in time:

$$\mathcal{K}g(x_k) = g(f(x_k))$$

Koopman Operator Theory

We look for special observables, called Koopman eigenfunctions $\varphi_j(x)$:

$$\mathcal{K}\varphi_j = \lambda_j\varphi_j$$

Koopman Operator Theory

We look for special observables, called Koopman eigenfunctions $\varphi_j(x)$:

$$\mathcal{K}\varphi_j = \lambda_j\varphi_j$$

The Koopman eigenfunctions form a basis for the space of observables:

$$g(\mathbf{x}) = \sum_j c_j \varphi_j(\mathbf{x})$$

- c_j : Expansion coefficients.

Koopman Operator Theory

We look for special observables, called Koopman eigenfunctions $\varphi_j(x)$:

$$\mathcal{K}\varphi_j = \lambda_j\varphi_j$$

The Koopman eigenfunctions form a basis for the space of observables:

$$g(\mathbf{x}) = \sum_j c_j \varphi_j(\mathbf{x})$$

The complex scalars λ_j are the corresponding Koopman eigenvalues. Then an observable g can be expanded:

$$\mathcal{K}^t g(\mathbf{x}) = \sum_j c_j \lambda_j^k \varphi_j(\mathbf{x})$$

- c_j : Expansion coefficients.
- φ_j : Koopman eigenfunction.
- λ_j : Koopman eigenvalues. Associated growth/decay/oscillation rate.

Dynamic Mode Decomposition (DMD)

The Challenge: $\mathcal{K}$ is infinite-dimensional, so finding a generally infinite number of its eigenfunctions and eigenvalues is not feasible in practice.

Dynamic Mode Decomposition (DMD)

The Challenge: $\mathcal{K}$ is infinite-dimensional, so finding a generally infinite number of its eigenfunctions and eigenvalues is not feasible in practice.

- DMD provides a practical, data-driven method to find a finite-dimensional approximation of the Koopman operator.

Dynamic Mode Decomposition (DMD)

The Challenge: $\mathcal{K}$ is infinite-dimensional, so finding a generally infinite number of its eigenfunctions and eigenvalues is not feasible in practice.

- DMD provides a practical, data-driven method to find a finite-dimensional approximation of the Koopman operator.
- Closely related to Koopman theory: linear evolution in function space

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

Dynamic Mode Decomposition (DMD)

The Challenge: $\mathcal{K}$ is infinite-dimensional, so finding a generally infinite number of its eigenfunctions and eigenvalues is not feasible in practice.

- DMD provides a practical, data-driven method to find a finite-dimensional approximation of the Koopman operator.
- Closely related to Koopman theory: linear evolution in function space

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

- DMD provides a modal decomposition based purely on data snapshots:

Snapshots Method (Lecture 3):

$$\begin{bmatrix} | & | & \dots & | \\ x_1 & x_2 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

Dynamic Mode Decomposition (DMD)

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

Dynamic Mode Decomposition (DMD)

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

$$X = \begin{bmatrix} | & | & \dots & | \\ x_1 & x_2 & \dots & x_{m-1} \\ | & | & & | \end{bmatrix} \quad \text{and} \quad X' = \begin{bmatrix} | & | & \dots & | \\ x_2 & x_3 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

Dynamic Mode Decomposition (DMD)

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

$$X = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \dots & x_{m-1} \\ | & | & & | \end{bmatrix} \quad \text{and} \quad X' = \begin{bmatrix} | & | & & | \\ x_2 & x_3 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

Assume approximate linear dynamics:

$$X' \approx AX \quad \Rightarrow \quad A \approx X'X^\dagger$$

Dynamic Mode Decomposition (DMD)

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

$$X = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \dots & x_{m-1} \\ | & | & & | \end{bmatrix} \quad \text{and} \quad X' = \begin{bmatrix} | & | & & | \\ x_2 & x_3 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

Assume approximate linear dynamics:

$$X' \approx AX \quad \Rightarrow \quad A \approx X'X^\dagger$$

The assumption is that there exists a **linear operator** A that advances the observables: $\mathbf{x}_{k+1} \approx A\mathbf{x}_k$.

$$\text{Best-fit matrix: } A = \arg \min_A \|X' - AX\|_F^2$$

Dynamic Mode Decomposition (DMD)

$$\mathbf{x}_{k+1} \approx A\mathbf{x}_k$$

$$X = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \dots & x_{m-1} \\ | & | & & | \end{bmatrix} \quad \text{and} \quad X' = \begin{bmatrix} | & | & & | \\ x_2 & x_3 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

Assume approximate linear dynamics:

$$X' \approx AX \quad \Rightarrow \quad A \approx X'X^\dagger$$

The assumption is that there exists a **linear operator** A that advances the observables: $\mathbf{x}_{k+1} \approx A\mathbf{x}_k$.

$$\text{Best-fit matrix: } A = \arg \min_A \|X' - AX\|_F^2$$

The solution is found via the Moore-Penrose pseudoinverse:

$$A = X'X^\dagger$$

Dynamic Mode Decomposition (DMD)

Perform SVD:

$$X = U\Sigma V^* \Rightarrow X \approx U_r \Sigma_r V_r^* \quad (\text{truncated to rank } r)$$

Dynamic Mode Decomposition (DMD)

Perform SVD:

$$X = U\Sigma V^* \Rightarrow X \approx U_r \Sigma_r V_r^* \quad (\text{truncated to rank } r)$$

Project onto low-dimensional subspace:

$$A = X'X^\dagger$$

$$\tilde{A} = U_r^* A U_r = U_r^* X' V_r \Sigma_r^{-1}$$

This is the DMD operator in reduced space.

DMD Modes and Eigenvalues

Eigen decomposition:

$$\tilde{A}W = W\Lambda \quad \Rightarrow \quad \Phi = X'V_r\Sigma_r^{-1}W$$

Project W back into the high-dimensional space

DMD Modes and Eigenvalues

Eigen decomposition:

$$\tilde{A}W = W\Lambda \quad \Rightarrow \quad \Phi = X'V_r\Sigma_r^{-1}W$$

Project W back into the high-dimensional space

- Φ : DMD modes (spatial structures)
- $\Lambda = \text{diag}(\lambda_k)$: DMD eigenvalues (how its corresponding mode grows, decays, and oscillates at each time step)

Reconstructing Dynamics

Initial amplitudes:

$$\mathbf{x}_1 = \sum b_k \phi_k \quad \Rightarrow \quad b = \Phi^\dagger \mathbf{x}_1$$

Reconstructing Dynamics

Initial amplitudes:

$$\mathbf{x}_1 = \sum b_k \phi_k \quad \Rightarrow \quad b = \Phi^\dagger \mathbf{x}_1$$

Time evolution:

$$\mathbf{x}(t) \approx \sum_{k=1}^r b_k \phi_k \lambda_k^{t-1}$$

Reconstructing Dynamics

Initial amplitudes:

$$\mathbf{x}_1 = \sum b_k \phi_k \quad \Rightarrow \quad b = \Phi^\dagger \mathbf{x}_1$$

Time evolution:

$$\mathbf{x}(t) \approx \sum_{k=1}^r b_k \phi_k \lambda_k^{t-1}$$

Matrix Form:

$$\mathbf{x}(t) \approx \Phi \text{diag}(\lambda)^{t-1} b$$

Reconstructing Dynamics

Initial amplitudes:

$$\mathbf{x}_1 = \sum b_k \phi_k \quad \Rightarrow \quad b = \Phi^\dagger \mathbf{x}_1$$

Time evolution:

$$\mathbf{x}(t) \approx \sum_{k=1}^r b_k \phi_k \lambda_k^{t-1}$$

Matrix Form:

$$\mathbf{x}(t) \approx \Phi \text{diag}(\lambda)^{t-1} b$$

- Φ : simple patterns in dynamical systems.
- $\text{diag}(\lambda)^{t-1}$: Evolves each mode's strength and phase forward $t - 1$ steps in time.
- b : Sets the starting strength of each mode.

Variants of DMD

- **Total Least Squares DMD:** Minimizes residuals in both X and X' (robust to noise)

Variants of DMD

- **Total Least Squares DMD:** Minimizes residuals in both X and X' (robust to noise)
- **DMD with Control (DMDc):**

$$X' = AX + BU \quad \text{Input-driven systems}$$

Variants of DMD

- **Total Least Squares DMD:** Minimizes residuals in both X and X' (robust to noise)
- **DMD with Control (DMDc):**

$$X' = AX + BU \quad \text{Input-driven systems}$$

- **Extended DMD (EDMD):**

$$X' = \mathcal{K}\Phi(X) \quad \text{Nonlinear observables } \Phi$$

Variants of DMD

- **Total Least Squares DMD:** Minimizes residuals in both X and X' (robust to noise)
- **DMD with Control (DMDc):**

$$X' = AX + BU \quad \text{Input-driven systems}$$

- **Extended DMD (EDMD):**

$$X' = \mathcal{K}\Phi(X) \quad \text{Nonlinear observables } \Phi$$

- **Streaming DMD:** Online update of modes as data arrive (memory efficient)

Variants of DMD

- **Total Least Squares DMD:** Minimizes residuals in both X and X' (robust to noise)
- **DMD with Control (DMDc):**

$$X' = AX + BU \quad \text{Input-driven systems}$$

- **Extended DMD (EDMD):**

$$X' = \mathcal{K}\Phi(X) \quad \text{Nonlinear observables } \Phi$$

- **Streaming DMD:** Online update of modes as data arrive (memory efficient)
- **DMD for Physical Systems:** piDMD (DMD with physical principles), giDMD (DMD with geometrical principles)

Sparse identification of nonlinear dynamics (SINDy)

Sparse identification of nonlinear dynamics (SINDy)

- Classical modeling: derive $\dot{\mathbf{x}} = f(\mathbf{x})$ from first principles.

Sparse identification of nonlinear dynamics (SINDy)

- Classical modeling: derive $\dot{\mathbf{x}} = f(\mathbf{x})$ from first principles.
- SINDy: discover governing equations directly from time-series data.

Sparse identification of nonlinear dynamics (SINDy)

- Classical modeling: derive $\dot{\mathbf{x}} = f(\mathbf{x})$ from first principles.
- SINDy: discover governing equations directly from time-series data.
- Promotes interpretability and sparsity: few terms govern dynamics.

$$\frac{d\mathbf{x}}{dt} \approx \Theta(\mathbf{x})\xi$$

Sparse identification of nonlinear dynamics (SINDy)

- 1 Collect time-series data $\mathbf{x}(t)$, estimate $\dot{\mathbf{x}}(t)$

Sparse identification of nonlinear dynamics (SINDy)

- 1 Collect time-series data $\mathbf{x}(t)$, estimate $\dot{\mathbf{x}}(t)$
- 2 Build a library $\Theta(\mathbf{x})$ of candidate nonlinear functions:

$$\Theta(\mathbf{x}) = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_1 x_2 & \cdots \end{bmatrix}$$

Sparse identification of nonlinear dynamics (SINDy)

- 1 Collect time-series data $\mathbf{x}(t)$, estimate $\dot{\mathbf{x}}(t)$
- 2 Build a library $\Theta(\mathbf{x})$ of candidate nonlinear functions:

$$\Theta(\mathbf{x}) = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_1 x_2 & \cdots \end{bmatrix}$$

- 3 Solve sparse regression problem:

$$\dot{\mathbf{x}} = \Theta(\mathbf{x})\xi$$

Formulating the Regression Problem

For each state variable:

$$\dot{x}_k(t) \approx \Theta(\mathbf{x}(t))\xi_k$$

We solve:

$$\xi_k = \arg \min_{\xi} \|\Theta\xi - \dot{x}_k\|_2^2 + \lambda\|\xi\|_0$$

$$\xi_k = \arg \min_{\xi} \|\Theta\xi - \dot{x}_k\|_2^2 + \lambda\|\xi\|_1$$

- λ : sparsity-promoting parameter
- Often solved via sequential thresholded least squares or LASSO

Formulating the Regression Problem

Sequential Thresholded Least Squares:

Formulating the Regression Problem

Sequential Thresholded Least Squares:

- 1 Solve standard least squares:

$$\xi = \arg \min_{\xi} \|\Theta \xi - \dot{x}\|_2^2$$

Formulating the Regression Problem

Sequential Thresholded Least Squares:

- 1 Solve standard least squares:

$$\xi = \arg \min_{\xi} \|\Theta \xi - \dot{x}\|_2^2$$

- 2 Threshold small coefficients:

$$\xi_i = 0 \quad \text{if } |\xi_i| < \lambda$$

Formulating the Regression Problem

Sequential Thresholded Least Squares:

- 1 Solve standard least squares:

$$\xi = \arg \min_{\xi} \|\Theta \xi - \dot{x}\|_2^2$$

- 2 Threshold small coefficients:

$$\xi_i = 0 \quad \text{if } |\xi_i| < \lambda$$

- 3 Repeat least squares on remaining active terms

Formulating the Regression Problem

Sequential Thresholded Least Squares:

- 1 Solve standard least squares:

$$\xi = \arg \min_{\xi} \|\Theta \xi - \dot{x}\|_2^2$$

- 2 Threshold small coefficients:

$$\xi_i = 0 \quad \text{if } |\xi_i| < \lambda$$

- 3 Repeat least squares on remaining active terms
- 4 Iterate until convergence

Formulating the Regression Problem

Sequential Thresholded Least Squares:

- 1 Solve standard least squares:

$$\xi = \arg \min_{\xi} \|\Theta \xi - \dot{x}\|_2^2$$

- 2 Threshold small coefficients:

$$\xi_i = 0 \quad \text{if } |\xi_i| < \lambda$$

- 3 Repeat least squares on remaining active terms
- 4 Iterate until convergence

Hyperparameter: Threshold $\lambda > 0$ controls sparsity level

SINDy Discovery: Governing Equation of 2D Oscillator

Simulated system:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -x_1 - x_1^3 \end{cases}$$

SINDy Discovery: Governing Equation of 2D Oscillator

Simulated system:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -x_1 - x_1^3 \end{cases}$$

Candidate library:

$$\Theta(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1 x_2 \quad x_2^2 \quad x_1^3]$$

SINDy Discovery: Governing Equation of 2D Oscillator

Simulated system:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -x_1 - x_1^3 \end{cases}$$

Candidate library:

$$\Theta(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1 x_2 \quad x_2^2 \quad x_1^3]$$

SINDy returns sparse coefficients:

$$\xi = \begin{bmatrix} 0 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & -1 \end{bmatrix}$$

Implicit SINDy

- Some physical systems cannot be sparsely represented in polynomial libraries.

Implicit SINDy

- Some physical systems cannot be sparsely represented in polynomial libraries.
 - **Rational dynamics:**

$$\dot{x} = \frac{x^2}{1+x}$$

Implicit SINDy

- Some physical systems cannot be sparsely represented in polynomial libraries.

- Rational dynamics:**

$$\dot{x} = \frac{x^2}{1+x}$$

- Conserved quantities, Hamiltonian dynamics

$$x^2 + y^2 = \text{constant}$$

$$\dot{q}_i = \frac{\partial H}{\partial p_i}, \quad \dot{p}_i = -\frac{\partial H}{\partial q_i}$$

Derivation: SINDy for Rational Dynamics

Step 1: Multiply by denominator

$$\dot{x} = \frac{x^2}{1+x} \Rightarrow (1+x)\dot{x} = x^2 \Rightarrow (1+x)\dot{x} - x^2 = 0$$

Derivation: SINDy for Rational Dynamics

Step 1: Multiply by denominator

$$\dot{x} = \frac{x^2}{1+x} \Rightarrow (1+x)\dot{x} = x^2 \Rightarrow (1+x)\dot{x} - x^2 = 0$$

Step 2: Rewriting as a sparse linear combination

$$x\dot{x} + \dot{x} - x^2 = 0$$

Derivation: SINDy for Rational Dynamics

Step 1: Multiply by denominator

$$\dot{x} = \frac{x^2}{1+x} \Rightarrow (1+x)\dot{x} = x^2 \Rightarrow (1+x)\dot{x} - x^2 = 0$$

Step 2: Rewriting as a sparse linear combination

$$x\dot{x} + \dot{x} - x^2 = 0$$

Step 3: Build joint function library

$$\Theta(x, \dot{x}) = [\dot{x} \quad x \quad x^2 \quad x\dot{x} \quad \dot{x}^2 \quad x^3 \quad x^2\dot{x}]$$

Derivation: SINDy for Rational Dynamics

Step 1: Multiply by denominator

$$\dot{x} = \frac{x^2}{1+x} \Rightarrow (1+x)\dot{x} = x^2 \Rightarrow (1+x)\dot{x} - x^2 = 0$$

Step 2: Rewriting as a sparse linear combination

$$x\dot{x} + \dot{x} - x^2 = 0$$

Step 3: Build joint function library

$$\Theta(x, \dot{x}) = [\dot{x} \quad x \quad x^2 \quad x\dot{x} \quad \dot{x}^2 \quad x^3 \quad x^2\dot{x}]$$

Step 4: Find sparse ξ :

$$\Theta(x, \dot{x})\xi = 0$$

Formulation of Implicit SINDy

Implicit SINDy seeks an equation of the form:

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) \boldsymbol{\xi} = 0$$

Formulation of Implicit SINDy

Implicit SINDy seeks an equation of the form:

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) \xi = 0$$

Construct a joint library:

$$\Theta(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_2^2 \quad x_1 x_2 \quad \cdots]$$

$$\Theta(\dot{\mathbf{x}}) = [1 \quad \dot{x}_1 \quad \dot{x}_2 \quad \dot{x}_1^2 \quad \dot{x}_2^2 \quad \dot{x}_1 \dot{x}_2 \quad \cdots]$$

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) = [1 \quad x_1 \quad x_2 \quad \dot{x}_1 \quad \dot{x}_2 \quad x_1^2 \quad x_1 \dot{x}_2 \quad \cdots]$$

Formulation of Implicit SINDy

Implicit SINDy seeks an equation of the form:

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) \xi = 0$$

Construct a joint library:

$$\Theta(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_2^2 \quad x_1 x_2 \quad \cdots]$$

$$\Theta(\dot{\mathbf{x}}) = [1 \quad \dot{x}_1 \quad \dot{x}_2 \quad \dot{x}_1^2 \quad \dot{x}_2^2 \quad \dot{x}_1 \dot{x}_2 \quad \cdots]$$

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) = [1 \quad x_1 \quad x_2 \quad \dot{x}_1 \quad \dot{x}_2 \quad x_1^2 \quad x_1 \dot{x}_2 \quad \cdots]$$

Goal: find sparse ξ such that equation holds at all data points

$$\min \|\xi\|_0 \quad \text{subject to } \Theta(\mathbf{x}, \dot{\mathbf{x}}) \xi = 0$$

Formulation of Implicit SINDy

Implicit SINDy seeks an equation of the form:

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) \xi = 0$$

Construct a joint library:

$$\Theta(\mathbf{x}) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_2^2 \quad x_1 x_2 \quad \cdots]$$

$$\Theta(\dot{\mathbf{x}}) = [1 \quad \dot{x}_1 \quad \dot{x}_2 \quad \dot{x}_1^2 \quad \dot{x}_2^2 \quad \dot{x}_1 \dot{x}_2 \quad \cdots]$$

$$\Theta(\mathbf{x}, \dot{\mathbf{x}}) = [1 \quad x_1 \quad x_2 \quad \dot{x}_1 \quad \dot{x}_2 \quad x_1^2 \quad x_1 \dot{x}_2 \quad \cdots]$$

Goal: find sparse ξ such that equation holds at all data points

$$\min \|\xi\|_0 \quad \text{subject to } \Theta(\mathbf{x}, \dot{\mathbf{x}}) \xi = 0$$

This is a null-space sparse regression problem.

SINDy Discovery: Governing Equation

True dynamics of Falling Ball:

$$\ddot{y} = -g - \alpha \dot{y} \quad \Rightarrow \quad a = -9.8 - \alpha v$$

SINDy Discovery: Governing Equation

True dynamics of Falling Ball:

$$\ddot{y} = -g - \alpha \dot{y} \quad \Rightarrow \quad a = -9.8 - \alpha v$$

Function Library

$$\Theta(y, v) = [1 \quad y \quad y^2 \quad v \quad v^2 \quad v^3]$$

SINDy Discovery: Governing Equation

True dynamics of Falling Ball:

$$\ddot{y} = -g - \alpha \dot{y} \quad \Rightarrow \quad a = -9.8 - \alpha v$$

Function Library

$$\Theta(y, v) = [1 \quad y \quad y^2 \quad v \quad v^2 \quad v^3]$$

SINDy discovers:

$$a(t) = \Theta(y(t), v(t))\xi \quad \text{with sparse } \xi$$

SINDy Discovery: Energy-Conserving System

True dynamics of Single Pendulum:

$$\ddot{\theta} + g \sin(\theta) = 0 \quad \Rightarrow \quad \text{energy conserved}$$

SINDy Discovery: Energy-Conserving System

True dynamics of Single Pendulum:

$$\ddot{\theta} + g \sin(\theta) = 0 \quad \Rightarrow \quad \text{energy conserved}$$

Function Library

$$\Theta(\theta, \omega) = [\sin \theta \quad \cos \theta \quad \omega \quad \omega^2 \quad \omega \sin \theta \quad \omega \cos \theta]$$

SINDy Discovery: Energy-Conserving System

True dynamics of Single Pendulum:

$$\ddot{\theta} + g \sin(\theta) = 0 \quad \Rightarrow \quad \text{energy conserved}$$

Function Library

$$\Theta(\theta, \omega) = [\sin \theta \quad \cos \theta \quad \omega \quad \omega^2 \quad \omega \sin \theta \quad \omega \cos \theta]$$

SINDy discovers:

$$E = \frac{1}{2} \dot{\theta}^2 + g(1 - \cos(\theta))$$

Discovering PDEs with SINDy

Goal: Discover PDEs of the form:

$$u_t = \mathcal{N}(u, u_x, u_{xx}, u_{xxx}, \dots)$$

from data $u(x, t)$

Discovering PDEs with SINDy

Goal: Discover PDEs of the form:

$$u_t = \mathcal{N}(u, u_x, u_{xx}, u_{xxx}, \dots)$$

from data $u(x, t)$

Approach:

- ① Simulate or collect data $u(x, t)$
- ② Compute $u_t, u_x, u_{xx}, u_{xxx}, \dots$
- ③ Build a library Θ of candidate terms
- ④ Solve sparse regression:

$$u_t \approx \Theta(u)\xi$$

Discovering PDEs with SINDy

Heat Equation:

$$u_t = \alpha u_{xx}$$

Discovering PDEs with SINDy

Heat Equation:

$$u_t = \alpha u_{xx}$$

- 1 Simulate $u(x, t)$ on a spatial grid
- 2 Estimate time derivative u_t and spatial derivatives
- 3 Build a library: $\Theta = [1, u, u^2, u_x, u_{xx}, u_{xxx}]$
- 4 Solve: $u_t \approx \Theta(u)\xi$ via sparse regression

Discovering PDEs with SINDy

Heat Equation:

$$u_t = \alpha u_{xx}$$

- ① Simulate $u(x, t)$ on a spatial grid
- ② Estimate time derivative u_t and spatial derivatives
- ③ Build a library: $\Theta = [1, u, u^2, u_x, u_{xx}, u_{xxx}]$
- ④ Solve: $u_t \approx \Theta(u)\xi$ via sparse regression

Expected output:

$$\xi = [0, 0, 0, 0, \alpha, 0]^T$$

$$u_t = \alpha u_{xx}$$

Challenges

- 1 Collect time-series data $\mathbf{x}$, estimate $\dot{\mathbf{x}}$, $\ddot{\mathbf{x}}$...
 - Noisy data
 - Methods to take derivatives

Challenges

- 1 Collect time-series data $\mathbf{x}$, estimate $\dot{\mathbf{x}}$, $\ddot{\mathbf{x}}$...
 - Noisy data
 - Methods to take derivatives
- 2 Build a library Θ of candidate nonlinear functions:

$$\Theta = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_1 x_2 & \cdots \end{bmatrix}$$

- How big the library is?
- What terms the library contains?

Challenges

① Collect time-series data $\mathbf{x}$, estimate $\dot{\mathbf{x}}$, $\ddot{\mathbf{x}}$...

- Noisy data
- Methods to take derivatives

② Build a library Θ of candidate nonlinear functions:

$$\Theta = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_1 x_2 & \cdots \end{bmatrix}$$

- How big the library is?
- What terms the library contains?

③ Solve sparse regression problem:

$$\dot{\mathbf{x}} = \Theta \xi$$

- Optimization algorithm
- Coding language

Summary of What You've Learned

- Convex Function Optimization
 - Gradient Descent Method

Summary of What You've Learned

- Convex Function Optimization
 - Gradient Descent Method
- Singular Value Decomposition
 - Matrix Completion (Low-rank Approximation)
 - Principal Component Analysis (High Dimensional Data)
 - Proper Orthogonal Decomposition (Time Series)
 - Latent Semantic Analysis (Natural Language Processing)

Summary of What You've Learned

- Convex Function Optimization
 - Gradient Descent Method
- Singular Value Decomposition
 - Matrix Completion (Low-rank Approximation)
 - Principal Component Analysis (High Dimensional Data)
 - Proper Orthogonal Decomposition (Time Series)
 - Latent Semantic Analysis (Natural Language Processing)
- Fourier Analysis
 - Feature Extraction (Time Series)

Summary of What You've Learned

- Convex Function Optimization
 - Gradient Descent Method
- Singular Value Decomposition
 - Matrix Completion (Low-rank Approximation)
 - Principal Component Analysis (High Dimensional Data)
 - Proper Orthogonal Decomposition (Time Series)
 - Latent Semantic Analysis (Natural Language Processing)
- Fourier Analysis
 - Feature Extraction (Time Series)
- Linear Regression
 - Linear Regression with/without Regularization
 - Compressed Sensing, Robust Principal Component Analysis

Summary of What You've Learned

- Convex Function Optimization
 - Gradient Descent Method
- Singular Value Decomposition
 - Matrix Completion (Low-rank Approximation)
 - Principal Component Analysis (High Dimensional Data)
 - Proper Orthogonal Decomposition (Time Series)
 - Latent Semantic Analysis (Natural Language Processing)
- Fourier Analysis
 - Feature Extraction (Time Series)
- Linear Regression
 - Linear Regression with/without Regularization
 - Compressed Sensing, Robust Principal Component Analysis
- Dynamical Systems
 - Fundamentals
 - Dynamic Mode Decomposition
 - Sparse Identification of Nonlinear Dynamics (SINDy, Model Discovery)